

Punas Power Shell (PPS)

Problem Statement & Project Vision

1. Problem Statement

Learning Linux commands traditionally requires access to a Linux operating system, WSL, a virtual machine, or a remote terminal environment. This creates a barrier for beginners who want to practice shell commands safely and understand how a real command-line environment works. Existing learning resources often explain commands separately from the environment in which they are used, making it difficult for learners to develop practical command-line skills.

Punas Power Shell (PPS) aims to address this problem by providing a shell environment that behaves like a Linux terminal while keeping the user's actual operating system safe. The project begins as a Python-based console shell called Punas Shell (PSH) and is designed to evolve into a browser-based Linux learning platform.

2. Project Objective

The primary objective is to develop Punas Shell (PSH), a modular command-line shell implemented from scratch using Python and its standard library. The shell will accept commands, parse them, execute the appropriate operations, maintain session state, and provide terminal-like output.

The long-term objective is to extend PPS into a web-based Linux simulation where users can practice commands without installing Linux or WSL. The web version will use a virtual filesystem containing mock files and directories and will simulate command behaviour safely.

3. Core Concept

Execute → Simulate → Teach

PPS should execute commands against its own controlled environment rather than merely displaying predefined examples. Normal commands should behave naturally. When a command has significant real-world risk or educational value, PPS can additionally explain what that command would do on an actual Linux system.

4. PPS Console – Version 1

- Interactive prompt: **psh>**
- Command parsing and dispatching
- File and directory operations
- Current working directory management
- Command history
- Logging of executed commands and filesystem activity
- Configuration management
- Error handling and validation

- Future support for pipes, redirection, wildcards, aliases, environment variables, and scripting

5. PPS Web – Version 2

- A clean browser interface containing the PPS/PSH terminal.
- A virtual Linux filesystem with mock files and folders.
- Commands execute against the virtual environment, never the user's real filesystem.
- All actions during the current web session are retained until the page is refreshed.
- Creating, deleting, renaming, moving, copying, and modifying files or directories changes the virtual filesystem.
- Command history and session state are retained during the active session.
- A refresh starts a fresh temporary session.
- Potential future support for saved/exported sessions.

6. Educational Safety Behaviour

The web version must never execute commands against the host computer. Commands that could be harmful on a real Linux system should instead be simulated inside PPS. After execution, PPS may provide a contextual explanation of the real-world effect.

Example:

```
psh> rm -rf /
```

Inside PPS, the command operates only on the virtual filesystem. PPS can then explain that on a real Linux system, recursively removing the root filesystem can destroy accessible system files and potentially make the operating system unusable. The user's real computer remains untouched.

7. Tutor and Learning Modes

- **Shell Mode:** Normal terminal-like behaviour with minimal additional explanation.
- **Tutor Mode:** Activated through commands such as **teach pps**, **teach psh**, or a future **tutor on** command. Commands continue to execute, but PPS explains each command after execution.
- **Interview Mode:** Provides Linux tasks without automatically revealing solutions or explanations.
- **Challenge/Quiz Mode:** Allows learners to practice commands and test their understanding.

8. State and Persistence Requirements

PPS must maintain the state of the current shell session. This includes the current directory, virtual files and directories, file contents, command history, environment variables, aliases, permissions when implemented, and learning-mode state.

Every meaningful user action should be recorded so that the shell can maintain a consistent state. This includes commands executed and filesystem changes such as file or folder creation, deletion, renaming, copying, moving, and modification.

9. Proposed Architecture

```
User
|
v
Punas Shell (PSH)
|
v
Command Parser
|
v
Command Dispatcher
|
v
PPS Session / Shell State
|
+--> Virtual / Local Filesystem
+--> Command History
+--> Environment
+--> Configuration
+--> Logger
|
v
Command Result / Output
```

10. Expected Learning Outcomes

- Understand how command-line shells work.
- Understand command parsing and dispatching.
- Understand filesystem concepts and directory hierarchies.
- Apply object-oriented programming and modular architecture.
- Practice exception handling, logging, configuration, and state management.
- Understand Linux command behaviour through safe simulation.
- Build a reusable shell engine that can support both CLI and web interfaces.

11. Long-Term Vision

PPS is intended to grow from a Python console application into an educational Linux shell platform. The same core concepts and command engine can eventually support PPS Console, PPS Learn, guided Linux labs, interview preparation, and other learning experiences.

Vision: Make Linux command-line learning accessible to anyone through a realistic, safe, and interactive shell experience without requiring a Linux installation or WSL.

12. Development Approach

Phase	Focus
Day 1	Framework and project structure

Day 2	Shell engine and interactive prompt
Day 3+	Basic commands and command dispatcher
Later	Filesystem, history, logging, configuration
Advanced	Pipes, redirection, wildcards, scripting
Future	PPS Web virtual Linux environment and learning modes

Project: Punas Power Shell (PPS) | Interactive shell: Punas Shell (PSH) | Document: Problem Statement & Vision